

Relatório Final — Computação em Nuvem

Projeto DijkFood

Pedro Tokar — João Felipe Vilas Boas — Vitor Matheus — Anderson Falcão

1. Introdução

O objetivo do projeto foi construir uma plataforma de delivery capaz de operar sob diferentes perfis de carga sem comprometer disponibilidade, latência e consistência do processamento. O contexto do problema exige a coexistência de componentes com características técnicas bastante diferentes: o cadastro administrativo do sistema demanda persistência relacional e consistência transacional; o rastreamento de entregadores requer alta taxa de atualização e baixa latência; e o cálculo de rotas demanda processamento intensivo em CPU e acesso eficiente a estruturas de grafo.

Para atender a esse objetivo, a solução foi dividida em quatro microsserviços independentes: **core-api**, **order-service**, **tracking-service** e **routing-service**. Essa separação foi guiada pela natureza do workload: operações CRUD e administrativas, processamento transacional de pedidos, rastreamento geoespacial com alta frequência de atualização e cálculo de rotas intensivo em CPU. A separação entre serviços também reduz acoplamento e evita que um aumento de carga em um subsistema afete diretamente os demais.

1.1. Objetivos específicos

- Implementar uma API REST para cadastro e consulta de usuários, restaurantes, entregadores e cardápio.
- Permitir a criação de pedidos com histórico completo e validação de transições de estado.
- Realizar cálculo de rotas sobre o grafo viário da cidade de São Paulo.
- Armazenar e consultar a posição de entregadores com baixa latência.
- Implantar a aplicação em nuvem com auto-scaling independente por serviço.
- Validar a arquitetura sob diferentes cenários de carga e falha.
- Consolidar uma análise de custos coerente com a solução implementada.

2. Arquitetura da solução

A solução adota um **Application Load Balancer** como ponto único de entrada. O ALB recebe as requisições externas e as encaminha por caminho para os serviços apropriados. O **core-api** concentra as operações de cadastro e consulta administrativa; o **order-service** coordena a criação e a progressão dos pedidos; o **tracking-service** mantém posição e disponibilidade dos entregadores; e o **routing-service** calcula rotas no grafo viário de São Paulo.

A decisão por microsserviços foi motivada por três fatores principais. O primeiro foi a necessidade de escalabilidade independente, uma vez que o cálculo de rotas e o rastreamento possuem comportamento diferente do CRUD administrativo. O segundo foi a necessidade de isolar responsabilidades de negócio, tornando o sistema mais organizado e mais fácil de testar. O terceiro foi a busca por flexibilidade tecnológica, permitindo que cada serviço utilizasse a estratégia de persistência mais adequada ao seu problema.

Serviço	Persistência	Responsabilidade principal
core-api	PostgreSQL + DynamoDB	CRUD de usuários, restaurantes, entregadores, menu e consulta administrativa de pedidos.
order-service	PostgreSQL	Criação do pedido, validação de transições de estado e registro do histórico do pedido.
tracking-service	DynamoDB	Atualização de posição, status operacional e busca geoespacial de entregadores.
routing-service	Grafo viário pré-processado	Cálculo de rotas, distâncias e tempos estimados entre origem e destino.

2.1. Princípios arquiteturais adotados

- **Separação por domínio:** cada serviço responde por um conjunto claro de responsabilidades.
- **Escalabilidade por necessidade:** cada componente cresce apenas quando sua própria carga aumenta.

- **Persistência adequada ao problema:** uso de banco relacional para dados transacionais e NoSQL para tracking.
- **Automação de infraestrutura:** provisionamento reprodutível com Terraform.
- **Baixo acoplamento operacional:** deploy e atualização independentes entre serviços.

Essa decomposição permitiu tratar cada componente de acordo com seu comportamento predominante. O **routing-service** recebeu mais CPU e memória por manter o grafo em memória e executar algoritmos de caminho mínimo; o **tracking-service** foi otimizado para escrita frequente e leitura por célula espacial; o **order-service** concentrou as regras de negócio transacionais; e a **core-api** permaneceu isolada para absorver grande volume de requisições de consulta e cadastro.

3. Fluxo funcional da aplicação

O fluxo de negócio principal começa quando um usuário consulta restaurantes e itens disponíveis, seleciona produtos e cria um pedido. A partir desse momento, o **order-service** valida o payload recebido, identifica um entregador disponível, registra o pedido e cria o primeiro evento de histórico. Em seguida, o sistema passa a acompanhar a evolução do pedido por meio de mudanças de estado, como criação, confirmação, preparação, coleta e entrega.

Em paralelo, o **tracking-service** mantém o estado atualizado dos entregadores, recebendo novas coordenadas geográficas e atualizando a célula espacial correspondente. Quando necessário, o **routing-service** é acionado para calcular a melhor rota entre pontos do processo logístico, por exemplo entre restaurante, entregador e cliente. A **core-api** expõe a visão administrativa desse ciclo, permitindo consultas, verificações e eventuais integrações externas.

4. Modelagem de dados e decisões técnicas

A persistência foi dividida entre PostgreSQL e DynamoDB. No PostgreSQL foram armazenadas as entidades com forte relacionamento e necessidade de integridade transacional: usuários, restaurantes, entregadores, estados do pedido, pedidos, eventos do pedido, itens e associação entre pedido e item. O histórico foi modelado em uma tabela específica de eventos, o que permitiu a consulta do histórico do pedido e permitiu que as atualizações de estado do pedido fossem sempre consistentes (um requisito importante de negócio), como será explicado na sessão dos serviços.

No DynamoDB foi criada a tabela CourierTracking, responsável por armazenar estado operacional do entregador, latitude, longitude, instante da última atualização e índice espacial H3. O uso de H3 foi importante para viabilizar consultas geoespaciais eficientes e compatíveis com a exigência de rastreamento de alta frequência, em particular o matching entre entregador e restaurante.

Conjunto de dados	Ferramenta	Motivação
Cadastros e pedidos	PostgreSQL	Relações bem definidas, integridade referencial, transações e histórico auditável.
Posição e status do entregador	DynamoDB	Baixa latência, alto volume de escrita e consulta por chave espacial.
Grafo viário	Arquivo pré-processado em memória	Leitura rápida e operação intensiva de algoritmo de rota.

4.1. Modelagem do banco de dados relacional

O esquema do banco de dados relacional, além de incluir as relações mais óbvias entre entidades (um pedido é associado a somente um usuário e um restaurante, um pedido tem vários itens, um item pertence a um restaurante e um restaurante tem vários itens, etc.), foi pensado para atender ao requisito de manter histórico do estado dos pedidos.

Para isso, além da tabela de pedidos, foi criada uma tabela de evento de pedidos, que é manipulada pelo serviço order-service (esta manipulação será melhor detalhada na sessão de implementação dos serviços). Essa tabela armazena as mudanças de estado de um pedido e em qual momento do tempo elas ocorreram, o que torna possível a leitura do histórico do pedido. Mesmo com a tabela de histórico, a tabela de pedidos também armazena o estado atual do pedido, o que permite que verificações do estado atual sejam feitas sem a necessidade de JOINS custosos.

No mais, o uso de um banco de dados relacional no lugar de uma solução como o DynamoDB ajuda a garantir a integridade das relações: a própria natureza e implementação do banco não permitirão que existam pedidos relacionados a clientes e restaurantes inexistentes.

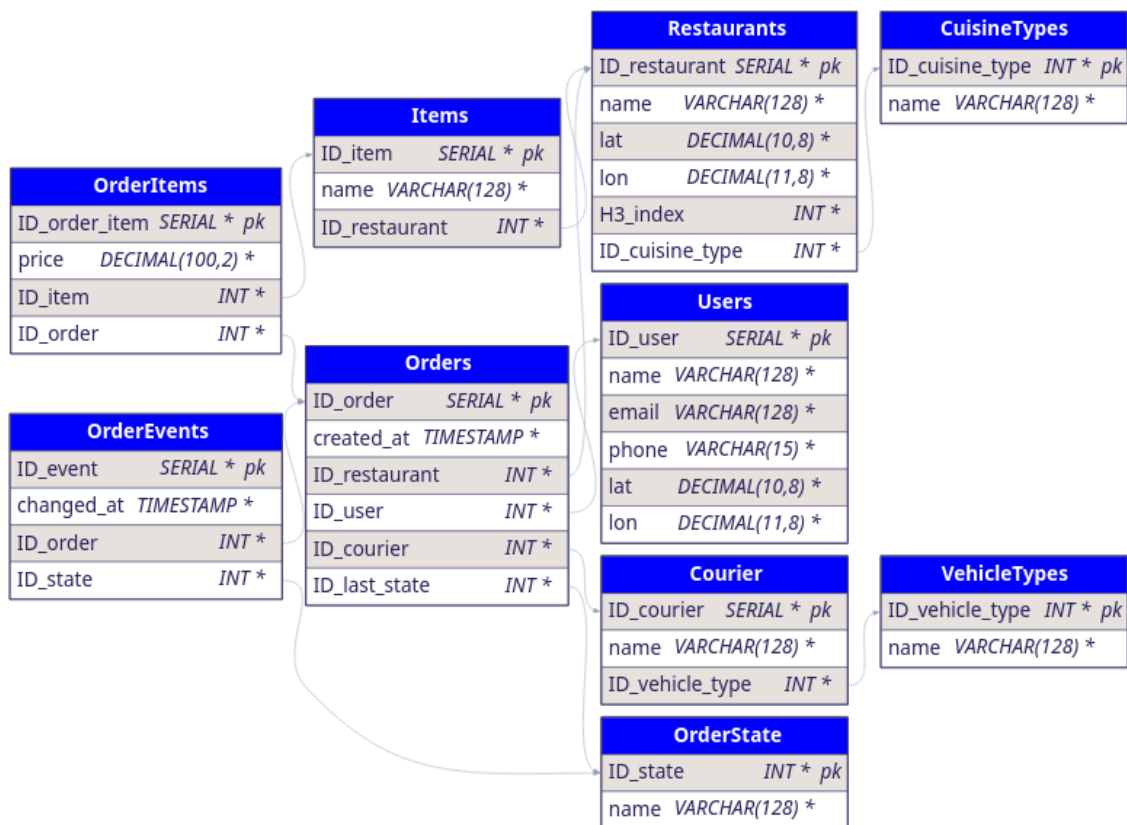


Figura 1: Esquema relacional usado no banco RDS.

4.2. Escolhas tecnológicas

As principais escolhas tecnológicas foram:

- **FastAPI** para expor as APIs REST com boa produtividade, validação tipada e desempenho.
- **SQLAlchemy assíncrono** nos serviços relacionais, permitindo integração natural com FastAPI e transações consistentes.
- **boto3** para integração com recursos AWS durante o deploy dos recursos.
- **aioboto3** para integração assíncrona com o DynamoDB nos serviços core-api e tracking-service.
- **OSMnx, NetworkX e BallTree** para cálculo de rotas sobre o grafo viário.
- **H3 Indexing** para matching entre loja e entregador.
- **Terraform e Docker** para infraestrutura como código e empacotamento reproduzível.

5. Implementação dos serviços

5.1. core-api

A **core-api** expõe as rotas de cadastro e consulta de usuários, restaurantes, entregadores e itens de menu. Também disponibiliza os endpoints administrativos para consulta do histórico de pedidos.

Um aspecto importante da implementação é que, no cadastro de entregadores, além de registrar os entregadores novos no banco de dados relacional, o sistema também cria uma entrada para o entregador no DynamoDB com latitude, longitude, célula H3 e status de disponibilidade. Quando uma nova requisição é feita, a API inicia uma transação no banco de dados relacional, verificando se o entregador existe no sistema. Em caso positivo, a API escreve a linha no DynamoDB e **só depois faz commit da transação no banco**. Dessa forma, não é possível haver um entregador no banco de dados relacional sem haver uma linha equivalente no DynamoDB. Isso é importante para evitar a observação de estados inconsistentes (um entregador não presente no DynamoDB iria quebrar o fluxo de pedidos).

5.2. order-service

O **order-service** concentra a lógica transacional do pedido (operações de criação e atualização de pedidos).

Na criação, o serviço recupera a localização do restaurante, seleciona um entregador disponível (com auxílio do **tracking-service**, que provê um endpoint para encontrar entregadores livre perto de uma localização) e define ele como ocupado no DynamoDB, registra o pedido no banco de dados com o estado inicial sendo o primeiro evento do histórico. O endpoint não escreve o pedido no banco caso haja falha para escrever o status do entregador como ocupado no DynamoDB (se um outro pedido foi atribuído para ele, por exemplo), o que evita que um entregador seja atribuído a dois pedidos de uma vez (um estado parcial indesejado).

A atualização de estado é feita em um endpoint que recebe o ID do pedido e o novo estado. Esse endpoint realiza uma validação do novo estado e, caso ele esteja dentro da ordem prevista, persiste o novo evento. Para que o requisito do estado do pedido não poder ser atualizado fora de ordem fosse atendido, a operação de checagem do estado do pedido (para ver se o novo estado é válido) e a operação de escrita no banco são feitas **em uma mesma transação do banco de dados relacional**. Como as operações de bancos relacionais são atômicas, não é possível que o estado do pedido se atualize por outro worker entre o processo de verificação e o de escrita.

```
WITH updated_order AS (  
  UPDATE Orders  
  SET ID_last_state = CAST(:id_novo_estado AS INTEGER)  
  WHERE ID_order = CAST(:id_pedido AS INTEGER)  
    AND ID_last_state = CAST(:id_estado_antigo_esperado AS INTEGER)  
  RETURNING ID_order, CAST(:id_novo_estado AS INTEGER) AS ID_state, CURRENT_TIMESTAMP AS  
  changed_at  
)  
  
INSERT INTO OrderEvents (ID_event, changed_at, ID_order, ID_state)  
SELECT  
  nextval('orderevents_id_event_seq'),  
  changed_at,  
  ID_order,  
  ID_state  
FROM updated_order  
RETURNING ID_order, ID_state;
```

Ao fazer um primeiro UPDATE com WHERE, a query se certifica de que a progressão correta dos estados será garantida, e o evento só será escrito se o primeiro UPDATE for bem sucedido. A atualização do estado do pedido também é responsável por liberar o entregador no DynamoDB quando ele chega ao fim, garantindo que um entregador não ficará sem poder receber novos pedidos. O commit da última transação só será feito se tiver sido possível liberar o entregador no DynamoDB, também para evitar estados parciais.

Esse serviço é central para o atendimento do trabalho porque materializa a exigência de histórico completo e auditável. Cada mudança importante gera um novo evento persistido, o que permite reconstruir a trajetória do pedido e produzir relatórios administrativos consistentes.

5.3. tracking-service

O **tracking-service** recebe atualizações de posição e status dos entregadores. Cada atualização recalcula a célula H3 do entregador e grava os novos valores na tabela de tracking, mantendo uma visão atualizada da frota em circulação. O serviço também implementa a consulta de proximidade, usada para localizar entregadores aptos a atender um restaurante ou pedido com baixa latência.

Do ponto de vista arquitetural, esse serviço foi isolado porque seu padrão de carga é muito diferente do restante do sistema. Enquanto o CRUD administrativo tende a ter requisições menos frequentes e mais previsíveis, o tracking pode receber muitas atualizações por segundo. Mesmo que as requisições sejam por sua natureza mais simples (o serviço basicamente replica a informação recebida no DynamoDB), elas são recebidas em uma escala muito maior do que os demais serviços.

5.4. routing-service

O **routing-service** carrega um grafo viário pré-processado de São Paulo e utiliza o algoritmo BallTree para encontrar os nós do grafo mais próximos de dadas coordenadas geográficas. A partir desses nós, o caminho é calculado com

Dijkstra bidirecional, retornando rota, distância e tempo estimado. Como esse serviço possui perfil intensivo em CPU e memória, sua separação em um componente dedicado permite que as características das máquina sejam diferentes das demais e também permite que o auto scaling seja feito por um trigger mais apropriado do que carga de networking.

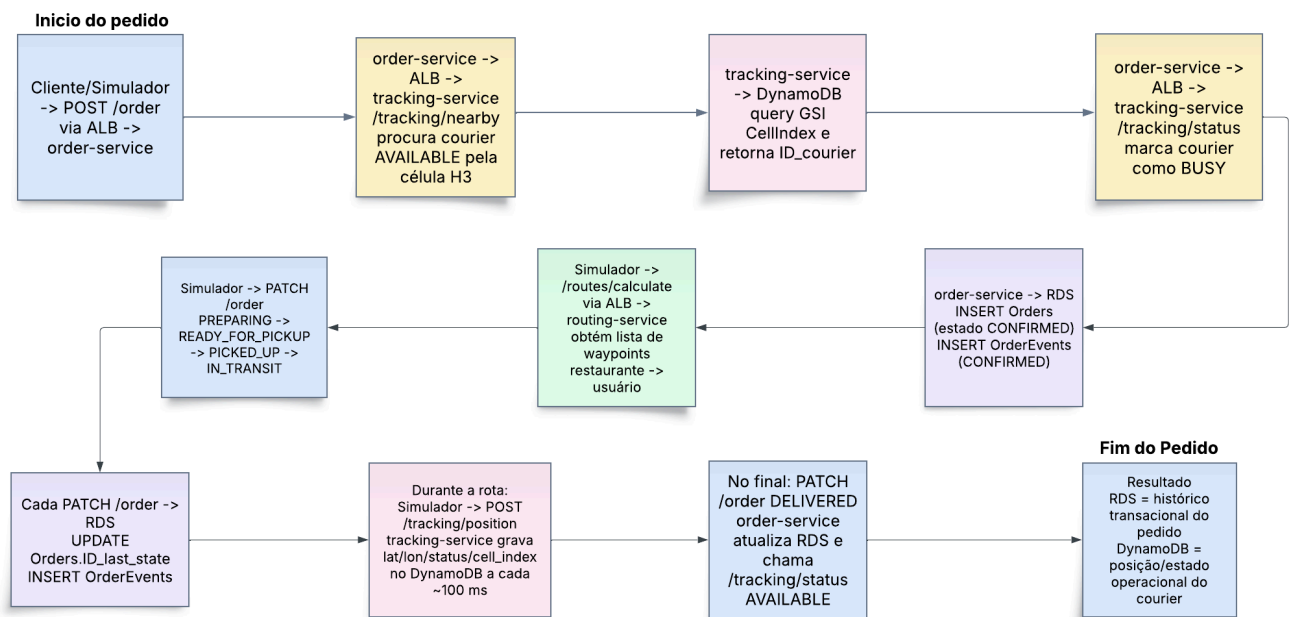


Figura 2: Fluxo do gerenciamento de pedidos.

6. Infraestrutura e implantação em nuvem

A infraestrutura foi descrita em Terraform e organizada em módulos independentes de VPC, ALB, ECS, RDS, DynamoDB, ECR. A rede foi estruturada com sub-redes públicas, privadas e de banco, distribuídas entre múltiplas zonas de disponibilidade. As tasks ECS executam em Fargate nas sub-redes privadas, enquanto o ALB permanece nas públicas como ponto único de entrada.

Cada serviço possui seu próprio **target group**, **task definition** e política de auto-scaling, permitindo crescimento independente conforme a demanda. O uso de Fargate eliminou a necessidade de administrar instâncias EC2, simplificando deploy, atualização e escalabilidade.

Serviço	Gatilho de escala	Justificativa
core-api	CPU + requisições por alvo no ALB	Absorver grande volume de consultas e cadastros sem impactar os demais componentes.
routing-service	CPU + requisições por alvo no ALB	Responder a bursts de cálculo de rota e workload intenso em CPU e memória.
tracking-service	CPU	Sustentar alta frequência de atualização com baixa latência.
order-service	CPU	Escalar conforme o volume de criação e progressão de pedidos.

6.1. Processo de deploy

O processo de implantação foi automatizado com Docker e um script de deploy que centraliza build das imagens, push para o ECR, aplicação do Terraform, inicialização do banco e **smoke tests** (testes simples para verificar o funcionamento básico dos serviços. No caso do script de deploy, todos os endpoints são testados para conferir se os serviços estão de fato em execução). Com isso, a solução passou a ser reproduzível e consistente entre ambiente local e nuvem.

De forma resumida, o pipeline de implantação segue as etapas abaixo:

- Deploy da infraestrutura definida no terraform para a AWS. Essa etapa precisa ser feita logo no começo, para garantir que os repositórios do ECR irão existir. Apesar dos clusters e serviços serem criados, eles ainda não ficam funcionais;
- Construção das imagens Docker dos serviços e upload delas no ECR, usando os comandos `docker build` e `docker push`, usando os links do ECR obtidos na etapa de provisionamento do terraform;
- Atualização forçada das tasks no ECR, para que elas possam de fato executar as imagens docker que foram colocadas no ECR. Após essa etapa, os serviços ficam disponíveis por meio do DNS do ALB;
- Envio, para testes no sistema, do script de simulação para uma máquina EC2 dedicada para execução dele.

A etapa de setup do esquema do banco de dados RDS, que é essencial para o funcionamento da API e para as garantias de atomicidade, é feita no serviço `core-api`. Ela usa os scripts de DDL e DML antes de iniciar o servidor HTTP, o que garante que o banco estará pronto para uso dela. Optamos por não incluir essa etapa diretamente no script de deploy para não ser necessário expor o banco para toda a internet ou ter que alterar dinamicamente as permissões de acesso dele.

A automação reduz erros manuais, melhora a repetibilidade e permite que o ambiente seja migrado de conta da AWS, caso seja necessário, ou até mesmo que seja replicado. Além disso, o uso de terraform permite que futuramente alguns componentes sejam migrados de plataforma provedora de nuvem (como por exemplo trocar o banco de dados para a Azure) sem que o script ou os serviços precisem sofrer grandes alterações.

Um resumo de toda essa arquitetura pode ser encontrado abaixo:

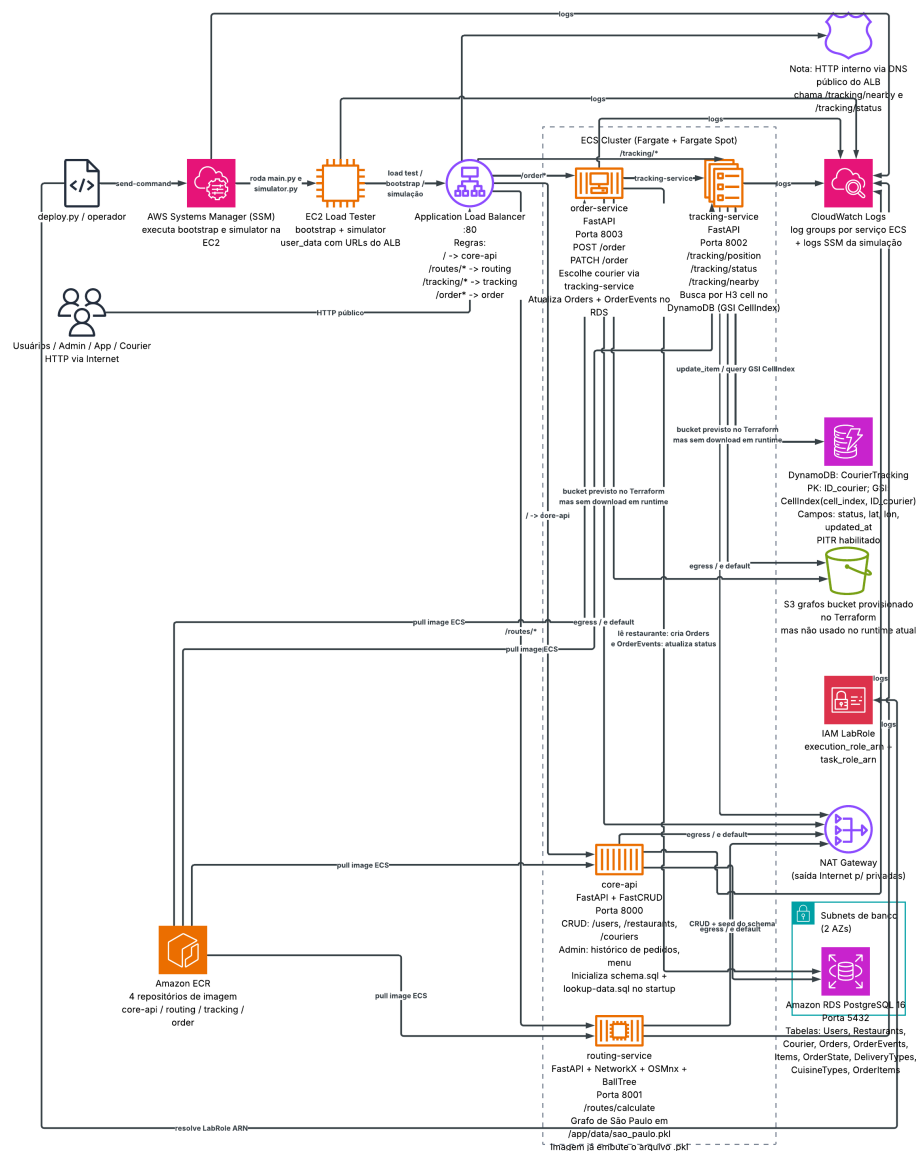


Figura 3: Esquema geral mostrando os recursos AWS utilizados e suas comunicações.

7. Tolerância a falhas, segurança e observabilidade

A tolerância a falhas foi tratada por meio da distribuição das tasks em múltiplas zonas de disponibilidade, uso de ALB para balanceamento de tráfego e políticas de reinicialização do ECS. Dessa forma, a falha de uma task ou a indisponibilidade pontual de uma instância lógica não interrompe o serviço como um todo, desde que haja ao menos uma réplica saudável para atendimento.

No aspecto de segurança, o sistema foi estruturado com separação entre sub-redes públicas e privadas, exposição apenas do balanceador de carga e isolamento dos serviços de aplicação dentro da rede privada. O banco relacional foi mantido em camada específica de rede, com acesso restrito aos componentes autorizados.

Quanto à observabilidade, a arquitetura foi pensada para permitir coleta de métricas, logs de aplicação e acompanhamento do comportamento de escalabilidade.

8. Resultados experimentais e atendimento aos requisitos

Foi realizada uma bateria de testes no dia anterior à apresentação parcial em aula do projeto. Esta bateria não visava medir os tempos de resposta da API, e sim testar as ferramentas de balanceamento automático do DynamoDB. Os testes foram realizados de diversos terminais em diversos computadores pessoais, e durante o teste foi possível observar o balanceamento automático em prática:

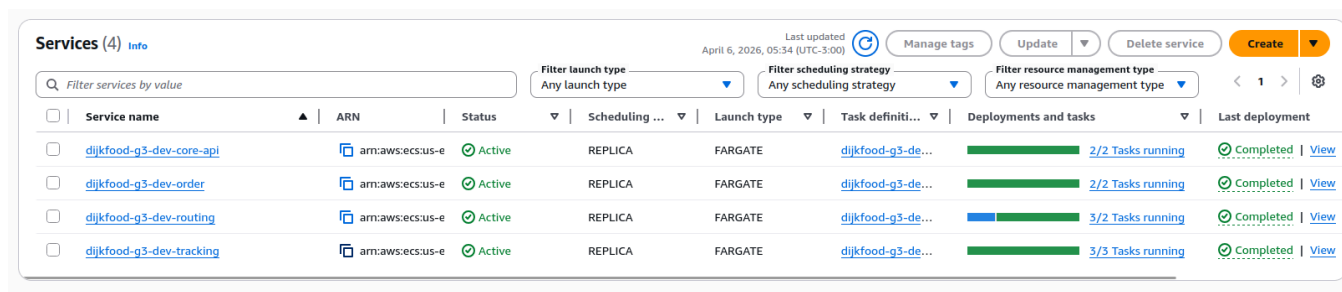


Figura 4: Quantidade de tasks antes do teste de carga

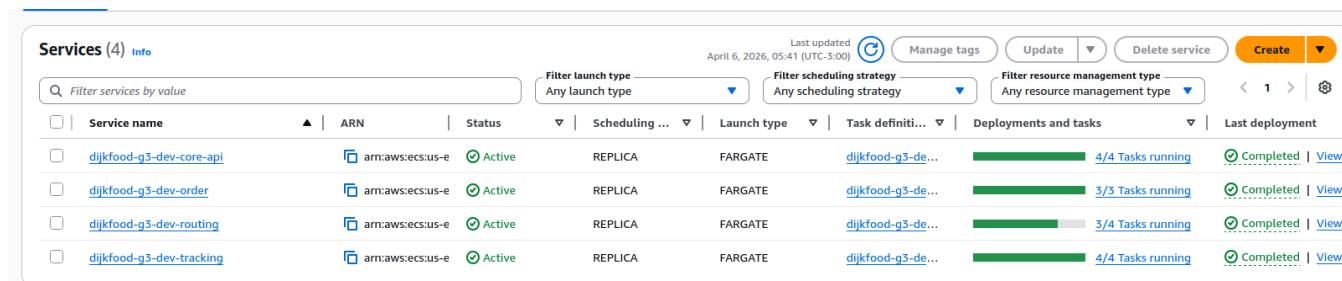


Figura 5: Quantidade de tasks depois do teste carga

O aumento do número de tasks sem intervenção manual foi importante para a validação da escolha de serviço. Além do número de tasks, também foi possível monitorar por meio do próprio console da AWS como se comportou o consumo de CPU das tasks (em conjunto). Observa-se que o aumento da carga faz o consumo médio das tasks aumentar, e também é possível ver que algumas tasks alcançam o consumo máximo de suas CPUs.

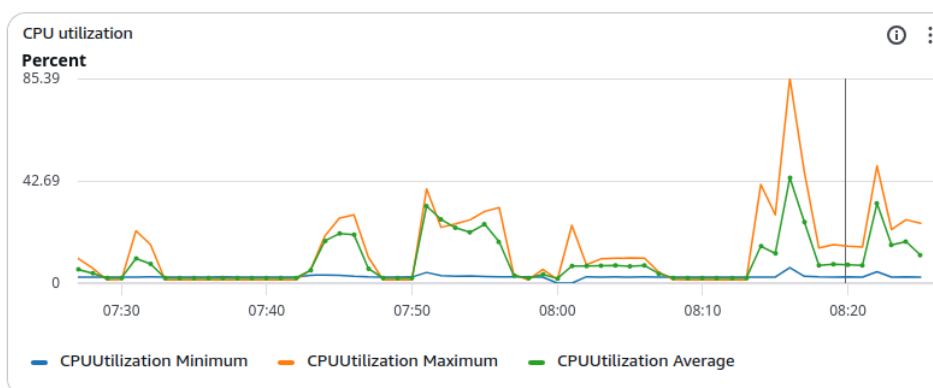


Figura 6: Consumo de CPU do cluster antes do autoscaling

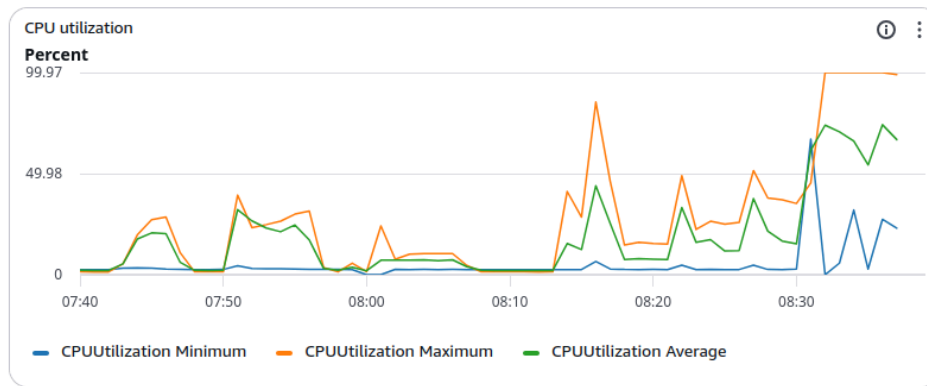


Figura 7: Consumo de CPU do cluster depois do autoscaling

Também foram realizados testes mais controlados, com apenas uma máquina disparando requisições de população do banco e de geração de pedidos. Eles foram executados em dois cenários: operação normal (10 pedidos por segundo) e pico (50 pedidos por segundo). O objetivo foi verificar throughput, latência por endpoint, comportamento do auto-scaling e isolamento entre componentes.

O resultado desses testes, porém, não foi semelhante ao que era esperado: algumas requisições sofreram de tempo de resposta altos, ao mesmo tempo em que as tasks não alcançavam o consumo de recursos suficiente para acionar a replicação delas. Esses resultados, apesar de desanimadores, deixam pistas de qual pode ser o problema do sistema e como resolver ele. Algumas possibilidades de melhoria e testes futuros incluem:

- Fazer testes para validar qual a melhor instância para o banco de dados relacional, e quais parâmetros dele (como `work_mem`) são ótimos;
- Fazer testes com a opção de autoscaling do DynamoDB (pode ajudar especialmente por conta do número alto de requisições de atualização de localização);
- Fazer testes com os parâmetros de unidade de leitura e unidade de escrita do DynamoDB;
- Fazer testes mais aprofundados para determinar, para cada serviço, quais os números mínimos e máximos de tasks ótimos e quais os melhores critérios e triggers para a alocação de mais tasks;
- Aprimorar a ferramenta de simulação de pedidos para registrar erros de API mais frequentes;
- Fazer testes unitários para capturar casos de mal funcionamento da lógica de negócio implementada na API REST.

A tabela a seguir mostra, para as simulações, qual foi o tempo médio de resposta dos endpoints de atualização de posição do entregador e de atualização do estado do pedido. A discrepância entre mediana e média, para as 3 primeiras linhas da tabela, mostra que o sistema ainda assim conseguiu responder grande parte das solicitações em tempo hábil, mas em compensação demorou tempos muito longos para responder as restantes. Devido aos resultados não satisfatórios no cenário de pico, não foram realizados testes controlados para o cenário de eventos (200 pedidos/s).

Cenário	Pedidos/s	Rota	Mediana	Média	Latência p95
Normal	10	Pedido	67,0 ms	120,1 ms	400,2 ms
Normal	10	Tracking	102,1 ms	3,3 s	25,7 s
Pico	50	Pedido	573,3 ms	2,1 s	8,0 s
Pico	50	Tracking	25,6 s	17,4 s	33,0 s

9. Análise de custo

A estimativa de custo foi construída na região `us-east-1`, considerando ECS Fargate, ALB, RDS, DynamoDB, NAT Gateway, e ECR. A diferença entre cenário normal e evento especial decorre principalmente do aumento do número médio de tasks nos serviços mais exigidos, especialmente **routing-service**, **core-api** e **order-service**. O custo fixo é dominado por ALB, NAT Gateway e base relacional; o custo variável cresce com o auto-scaling e com o volume de requisições processadas.

O gráfico abaixo foi retirado do dashboard de custos da AWS, para um sistema que ficou em deploy durante aproximadamente duas semanas (**sem** uma EC2 de simulação).

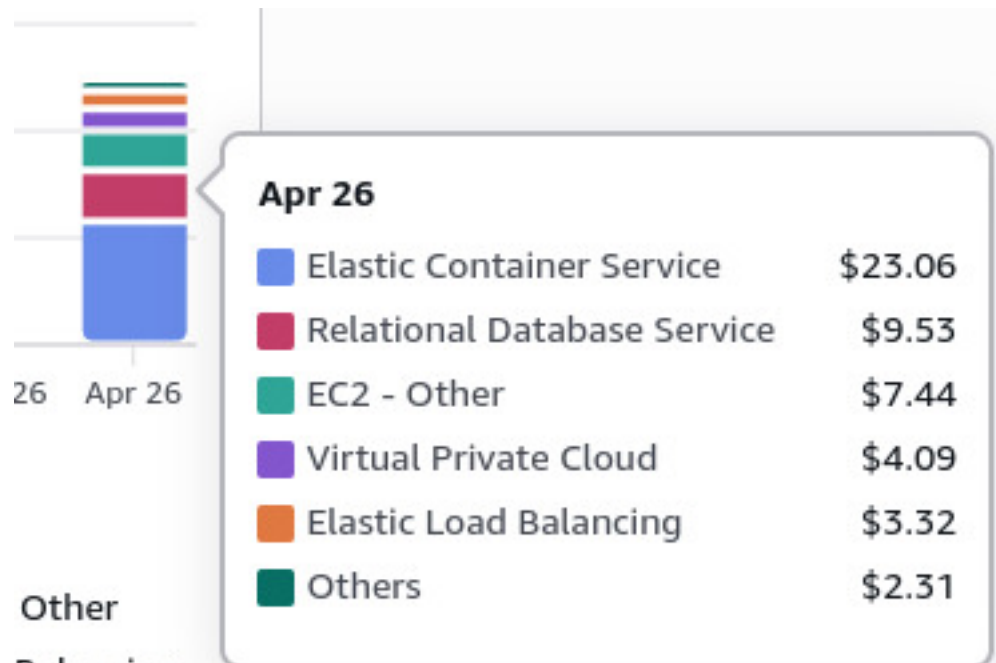


Figura 8: Custo Por serviço.